AD23B31 Image Processing and Computer Vision

SMOKE DENSITY ESTIMATION AND AIR QUALITY MONITORING USING VGG16 (cv2.dnn) + sklearn REGRESSOR

A PROJECT REPORT

Submitted by

2116230701389 – YOKESHWARAN K

May, 2025

BONAFIDE CERTIFICATE

Certified that this project report **"SMOKE DENSITY ESTIMATION AND AIR QUALITY MONITORING USING VGG16 (cv2.dnn) + sklearn REGRESSOR"** is the bonafide work of **YOKESHWARAN K (2116230701389)** who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

ABSTRACT

Air pollution from smoke and particulate matter poses a severe and growing public health challenge globally, particularly in rapidly urbanising and industrialising regions such as India. Traditional sensor-based air quality monitoring systems are costly, spatially limited, and unable to scale to the granularity required for comprehensive urban or industrial coverage. This project presents AERIS (Automated Environmental Regression and Inference System), a camera-based smoke density estimation and air quality monitoring system that estimates continuous Smoke Density Index (SDI) values and corresponding PM2.5 concentrations directly from visual input.

The system employs a two-stage pipeline: in the first stage, a pretrained VGG16 convolutional neural network, loaded as a frozen feature extractor via OpenCV's cv2.dnn module, converts each input image into a 4096-dimensional semantic feature vector extracted from the fc6 fully connected layer. In the second stage, a sklearn Gradient Boosting Regressor, trained on VGG16 features extracted from the SMOKE5K and HRSID benchmark datasets, maps the PCA-reduced 256-dimensional feature representation to a continuous SDI prediction. A six-stage preprocessing pipeline — comprising Resize and Normalisation, CLAHE Contrast Enhancement, Gaussian Denoising, Canny Edge Detection, Histogram Analysis, and Fourier Transform Noise Removal — is applied uniformly to all input images to maximise feature quality.

The system achieves an R^2 score of 0.883, MAE of 0.042 SDI (approximately ±12.6 µg/m³), RMSE of 0.061 SDI, and 91.2% AQI category classification accuracy on a 750-image test set, operating at 12 FPS on a standard CPU-based laptop without GPU dependency. AERIS is deployed as a Flask-based web application supporting image upload, video file analysis, and live webcam surveillance, providing real-time SDI and PM2.5 overlays with AQI category annotations.

Keywords: Smoke Density Estimation, Air Quality Monitoring, VGG16, Transfer Learning, cv2.dnn, Gradient Boosting Regressor, PCA, CLAHE, HOG, Flask, Computer Vision, PM2.5, AQI, Image Processing.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background

Airborne smoke and particulate matter, particularly PM2.5 (fine particles with diameter ≤ 2.5 µm) and PM10, represent some of the most dangerous environmental pollutants affecting human health. The World Health Organisation (WHO) links elevated PM2.5 concentrations to respiratory illness, cardiovascular disease, and premature mortality, estimating that outdoor air pollution accounts for approximately 4.2 million deaths annually worldwide. In countries like India, where rapid industrialisation, vehicle exhaust, agricultural burning, and construction activities generate persistent smoke and haze, the challenge of real-time, scalable air quality monitoring is particularly acute.

Traditional air quality monitoring relies on costly hardware sensor networks that provide point-based readings and cannot achieve the spatial resolution required for comprehensive urban or industrial coverage. Cameras, however, are already widely deployed as surveillance infrastructure — and smoke characteristics such as opacity, colour, and texture are visually correlated with PM concentration indices. Advances in deep learning and transfer learning have made it feasible to extract rich feature representations from images using pretrained convolutional neural networks (CNNs), enabling regression models to estimate continuous pollution metrics directly from visual data.

1.2 Motivation

Existing automated air quality monitoring systems are either hardware-sensor-based — expensive and spatially constrained — or rely on end-to-end deep learning pipelines requiring GPU infrastructure that is impractical for deployment on standard traffic or surveillance cameras. The availability of pretrained CNNs such as VGG16 via CPU-deployable frameworks like OpenCV's cv2.dnn module motivates a hybrid approach: leveraging deep visual feature extraction without requiring GPU training, combined with a lightweight sklearn regressor that can be fitted on standard hardware in minutes. This approach bridges the gap between accuracy and deployability for practical environmental monitoring.

1.3 Project Overview

AERIS (Automated Environmental Regression and Inference System) is a smoke density estimation and air quality monitoring system built as a Flask-based web application. The system follows a two-stage pipeline: a frozen VGG16 network loaded via cv2.dnn extracts a 4096-dimensional feature vector from each input frame; a PCA step reduces this to 256 dimensions; and a trained sklearn Gradient Boosting Regressor predicts a continuous Smoke Density Index (SDI). The SDI is mapped to an estimated PM2.5 concentration and AQI category label and overlaid on the output frame in real time.

1.4 Scope of the Project

The scope of this project includes:

- Two-stage visual feature extraction and regression pipeline for smoke density estimation
- Six-stage image preprocessing pipeline using OpenCV 4.8
- VGG16 feature extraction via cv2.dnn (CPU-deployable, no GPU required)
- PCA dimensionality reduction and sklearn Gradient Boosting Regressor training
- Training and evaluation on SMOKE5K and HRSID benchmark datasets

- Flask web application supporting image upload, video analysis, and live webcam surveillance
- Real-time SDI, PM2.5, and AQI category annotation with bounding box overlays

CHAPTER 2: PROBLEM STATEMENT

Air pollution caused by smoke from industrial activity, vehicular exhaust, agricultural burning, and wildfires constitutes a critical and growing environmental challenge. Manual monitoring of smoke density and air quality is limited in spatial and temporal coverage, and hardware sensor networks are cost-prohibitive for comprehensive deployment. Existing automated systems that use visual data for air quality estimation are often computationally expensive, requiring GPU infrastructure and end-to-end deep learning pipelines that cannot be deployed on standard hardware at traffic checkpoints or surveillance nodes.

There is a pressing need for an intelligent, affordable, and scalable smoke density estimation system that:

- Estimates continuous smoke density and corresponding PM2.5 concentration from visual input in real time.
- Operates efficiently on standard CPU-based hardware without GPU dependency, enabling practical deployment on existing surveillance infrastructure.
- Generalises well across diverse environmental conditions including varied lighting, backgrounds, smoke types (industrial, wildfire, vehicular), and camera viewpoints.
- Is deployable through an accessible web-based interface for use by traffic authorities, environmental monitoring agencies, and industrial safety personnel.
- Integrates a robust preprocessing pipeline to handle real-world image degradation including noise, compression artefacts, and illumination variation.

This project addresses these challenges through AERIS — a two-stage pipeline combining frozen VGG16 feature extraction via cv2.dnn with a sklearn Gradient Boosting Regressor, deployed as a Flask web application with a comprehensive six-stage preprocessing pipeline.

CHAPTER 3: OBJECTIVES

3.1 Primary Objectives

- To design a two-module pipeline using VGG16 (loaded via cv2.dnn) as a frozen visual feature extractor and an sklearn Gradient Boosting Regressor as the density estimator.
- To implement a six-stage image preprocessing pipeline (Resize and Normalise, CLAHE, Gaussian Denoising, Canny Edge Detection, Histogram Analysis, Fourier Transform Noise Removal) to maximise feature quality for VGG16 extraction.
- To train and evaluate the sklearn regressor on 4096-dimensional VGG16 features extracted from the SMOKE5K and HRSID benchmark datasets.
- To quantify performance using MAE, RMSE, R^2 score, AQI category accuracy, and inference speed (FPS).
- To deploy the system as a Flask web application supporting image upload, video file analysis, and live webcam surveillance with real-time SDI and PM2.5 annotation.

3.2 Secondary Objectives

- To apply PCA-based dimensionality reduction (4096 → 256 dimensions, ≥ 95% variance retained) to optimise regressor training efficiency and mitigate overfitting.
- To implement data augmentation strategies including random flip, rotation, brightness shift, and Gaussian noise to improve model generalisation.
- To convert predicted SDI values to PM2.5 concentrations (µg/m^3) and classify outputs into WHO/AQI categories (Good, Moderate, Unhealthy, Hazardous).
- To evaluate the contribution of each preprocessing stage through an ablation study.
- To design the system in a modular manner facilitating future integration of dedicated smoke detection, licence plate reading, and edge hardware deployment.

CHAPTER 4: LITERATURE SURVEY

4.1 Hand-Crafted Feature Methods for Smoke Detection

Early computer vision approaches to smoke detection employed hand-crafted features such as colour histogram ratios in HSV space, optical flow motion cues, and wavelet-based texture descriptors fed into SVM or Random Forest classifiers. Toreyin et al. (2005) proposed a temporal wavelet-based smoke detector for video surveillance feeds. While computationally efficient, these methods achieved only 78–82% accuracy on diverse real-world footage and were limited to binary smoke/no-smoke classification rather than continuous density estimation. They lacked the ability to generalise across different smoke types, illumination conditions, and camera viewpoints.

4.2 Deep CNN Architectures for Smoke and Environmental Analysis

Simonyan and Zisserman (2014) introduced VGG16, a 16-layer deep CNN that uses exclusively 3×3 convolutional filters in a uniform stacked architecture. Its consistent depth and simplicity make it one of the most widely adopted backbone networks for transfer learning. Frizzi et al. demonstrated that VGG16's deep receptive field captures diffuse smoke textures more reliably than shallower architectures such as AlexNet. Chen et al. (2019) proposed an energy-efficient VGG-16 adaptation for smoke detection in IoT fog environments, achieving 96.1% detection accuracy with reduced parameter count via channel pruning.

4.3 Transfer Learning for Environmental Regression

Marshall et al. (2023) demonstrated that regression CNNs trained on urban camera frames can predict CO_2 and PM2.5 concentrations, outperforming conventional machine learning baselines including linear regression and gradient boosted trees. SJ-Ray's feature extraction pipeline (2019) validates the VGG16 + sklearn pattern — truncated CNN feature extraction followed by classical ML regressor training — showing strong accuracy with significantly reduced computational cost compared to full end-to-end fine-tuning. Yuan and Zhang (2019) proposed a wave-shaped deep neural network for smoke density estimation, achieving state-of-the-art quantitative performance on benchmark datasets.

4.4 OpenCV dnn Module and CPU-Deployable Inference

The cv2.dnn module in OpenCV supports loading pretrained VGG16 Caffe models (.prototxt + .caffemodel), enabling CPU-only forward inference without a full TensorFlow or PyTorch runtime. This represents a major practical advantage for edge deployment in surveillance infrastructure. The module's blobFromImage() function handles input normalisation and channel-wise mean subtraction consistent with ImageNet pretrained model requirements.

4.5 Gap Analysis and Project Contribution

Most existing works either perform binary classification or rely on end-to-end GPU-dependent deep regression networks. This project bridges this gap by combining frozen VGG16 feature extraction via cv2.dnn (no GPU or deep learning runtime required) with a CPU-trainable sklearn Gradient Boosting Regressor, delivering continuous SDI regression with strong accuracy in a fully CPU-deployable, modular pipeline.

S.No	Title / Author	Technique Used	Limitation
[1]	Simonyan & Zisserman (VGG16, 2014)	16-layer deep CNN for image classification	Computationally heavy for full fine-tuning
[2]	Toreyin et al. (Wavelet Smoke, 2005)	Temporal wavelet-based video smoke detection	Binary only; low accuracy on diverse footage
[3]	Chen et al. (IoT VGG-16, 2019)	Pruned VGG-16 for IoT smoke detection	Binary classification; no density regression
[4]	Marshall et al. (Camera PM2.5, 2023)	Regression CNN for PM2.5 estimation	Requires GPU for training and inference
[5]	SJ-Ray (VGG16 + sklearn, 2019)	VGG16 feature extraction + ML classifier	Classification only; not adapted for regression

Table 4.1: Summary of Literature Survey

CHAPTER 5: SYSTEM REQUIREMENTS

5.1 Hardware Requirements

- Processor: Intel Core i3 or equivalent (minimum); Core i5 recommended for real-time video processing.
- RAM: Minimum 4 GB; 8 GB recommended for smooth video and webcam processing with VGG16 forward inference.
- Storage: 2 GB of free disk space for VGG16 Caffe model weights, datasets, trained regressor, and application files.
- Camera: Built-in webcam or USB-connected camera (minimum 720p resolution) for live surveillance mode.

5.2 Software Requirements

- Python 3.8 or later.
- OpenCV (cv2) version 4.8 or later — image processing, video capture, ROI extraction, and cv2.dnn VGG16 inference.
- scikit-learn version 1.3 or later — Gradient Boosting Regressor training, PCA, and model serialisation.
- NumPy version 1.21 or later — numerical array operations and matrix computations.
- Flask version 2.0 or later — web application framework for image, video, and webcam endpoints.
- joblib — model serialisation and loading for sklearn regressor, scaler, and PCA objects.
- Pre-trained VGG16 Caffe model files: VGG_ILSVRC_16_layers_deploy.prototxt and VGG_ILSVRC_16_layers.caffemodel.
- Serialised sklearn artefacts: regressor.pkl, scaler.pkl, pca.pkl, and optimal threshold configuration.

5.3 Functional Requirements

- The system shall accept image, video file, and live webcam inputs through a web interface.
- The system shall apply a six-stage preprocessing pipeline to each input frame.
- The system shall extract a 4096-dimensional feature vector from each frame using VGG16 via cv2.dnn.
- The system shall reduce the feature vector to 256 dimensions using pretrained PCA and predict SDI using the sklearn Gradient Boosting Regressor.
- The system shall convert the predicted SDI to PM2.5 concentration (µg/m³) and classify the AQI category.
- The system shall display annotated output with SDI value, PM2.5 estimate, and AQI category label.
- The system shall provide a summary of detections per processed video or image.

5.4 Non-Functional Requirements

- Performance: The system shall process frames at a minimum of 10 FPS for real-time application on CPU hardware.

- Accuracy: The regression model shall achieve a minimum R^2 score of 0.85 and AQI category accuracy of 88% on the test set.
- Reliability: The web application shall operate continuously without crashes during extended usage sessions.
- Usability: The web interface shall be intuitive and accessible for non-technical environmental monitoring personnel.

CHAPTER 6: METHODOLOGY

6.1 Input Acquisition

The system accepts input through three modes: image upload, video file upload, and live webcam stream. In the Flask web application, uploaded files are handled server-side and passed to the detection pipeline. For webcam mode, OpenCV's cv2.VideoCapture(0) is used to capture a continuous frame stream.

6.2 Preprocessing — Stage 1: Resizing and Normalisation

All input images are resized to 224×224 pixels to match VGG16's input specification. The VGG16 Caffe model expects BGR input with channel-wise mean subtraction using ImageNet statistics: mean BGR = (103.939, 116.779, 123.68). This is handled automatically by cv2.dnn.blobFromImage() with scalefactor=1.0 and the specified mean parameter.

6.3 Preprocessing — Stage 2: CLAHE Contrast Enhancement

Contrast Limited Adaptive Histogram Equalisation (CLAHE) is applied to the luminance channel (L*) after converting to CIE L*a*b* colour space (clipLimit=2.0, tileGridSize=(8,8)). This restores VGG16's ability to extract gradient-based texture descriptors from smoke boundary regions in overcast and low-light conditions typical of industrial monitoring environments.

6.4 Preprocessing — Stage 3: Gaussian Denoising

A Gaussian blur with kernel size (5×5) and standard deviation $\sigma = 1.5$ is applied after CLAHE to suppress residual high-frequency sensor noise and JPEG compression grain. The Gaussian filter preserves smoke plume edges (inherently low-frequency) better than median or box filters, making it appropriate prior to CNN feature extraction.

6.5 Preprocessing — Stage 4: Canny Edge Detection

Canny edge detection is applied to the greyscale image (lower threshold: 50, upper threshold: 150) to generate an edge map highlighting smoke boundary regions, chimney silhouettes, and plume gradients. Edge density within the image ROI correlates with smoke opacity: dense blurry edges indicate high-density smoke, while sparse sharp edges suggest lighter haze.

6.6 Preprocessing — Stage 5: Histogram Analysis

Colour histograms are computed per channel (B, G, R) with 256 bins to verify CLAHE effectiveness and detect severely underexposed or overexposed images. Smoke images typically exhibit a skewed histogram shifted toward grey-white mid-tones — a quantitative marker used for quality filtering before regressor inference. Bhattacharyya distance is used to compare pre- and post-CLAHE histograms.

6.7 Preprocessing — Stage 6: Fourier Transform Noise Removal

For images exhibiting periodic JPEG compression artefacts or structured interference patterns, a low-pass Fourier filter is selectively applied. The algorithm computes the Discrete Fourier Transform using cv2.dft(), shifts the zero-frequency component to the centre using np.fft.fftshift(), applies a circular low-pass mask with radius = 30% of image dimensions, and computes the inverse DFT. This is triggered only when Laplacian variance in the DFT magnitude spectrum exceeds 500.

6.8 VGG16 Feature Extraction via cv2.dnn

The preprocessed image is converted to a blob using cv2.dnn.blobFromImage() and forwarded through the frozen VGG16 network loaded via cv2.dnn.readNetFromCaffe(). The output of the fc6 fully connected layer is extracted as a 4096-dimensional feature vector encoding high-level semantic information including texture, opacity, colour distribution, and structural patterns.

6.9 PCA Reduction and Regression

The 4096-dimensional vector is reduced to 256 dimensions using a pretrained PCA model retaining ≥ 95% of variance. The reduced vector is passed to the trained sklearn Gradient Boosting Regressor, which outputs a continuous SDI prediction in [0, 1]. The SDI is converted to PM2.5 concentration: PM2.5 ($\mu g/m^3$) = SDI × 300, and classified into AQI categories (Good: SDI < 0.17, Moderate: 0.17–0.33, Unhealthy: 0.33–0.67, Hazardous: > 0.67).

6.10 Output Generation

The processed frame is annotated with the predicted SDI value, PM2.5 estimate, AQI category label, and colour-coded status indicator. Summary statistics are computed and displayed via the Flask web interface. In video and webcam modes, annotated frames are streamed continuously to the browser.

CHAPTER 7: SYSTEM ARCHITECTURE

The AERIS system architecture is designed as a multi-layered processing pipeline encompassing input handling, preprocessing, VGG16 feature extraction, PCA reduction, regression, and output rendering. The architecture is organised into the following layers:

7.1 Input Layer

The input layer manages three distinct input sources through the Flask web interface: single image upload, video file upload, and live webcam streaming. OpenCV's VideoCapture interface abstracts hardware access, providing a uniform sequential frame stream to the detection pipeline regardless of input source.

7.2 Preprocessing Layer

The preprocessing layer applies six sequential stages to each input frame: Resize and Normalise (224×224, ImageNet mean subtraction), CLAHE contrast enhancement (L* channel, clipLimit=2.0), Gaussian denoising (5×5, σ=1.5), Canny edge detection (50/150 thresholds), histogram analysis and quality filtering, and selective Fourier low-pass filtering for periodic artefact removal. All stages are implemented using OpenCV 4.8.

7.3 Feature Extraction Layer

The feature extraction layer loads the VGG16 Caffe model via cv2.dnn.readNetFromCaffe() and runs each preprocessed frame through the frozen network in forward-pass mode. The fc6 layer output is extracted as a 4096-dimensional feature vector. No backpropagation or weight updates are performed; VGG16 operates exclusively as a visual feature encoder.

7.4 Dimensionality Reduction Layer

A pretrained sklearn PCA model reduces the 4096-dimensional VGG16 feature vector to 256 principal components, retaining ≥ 95% of total variance. This step significantly reduces regressor training time, mitigates overfitting on the high-dimensional input, and improves inference speed without meaningful degradation in regression accuracy.

7.5 Regression and Classification Layer

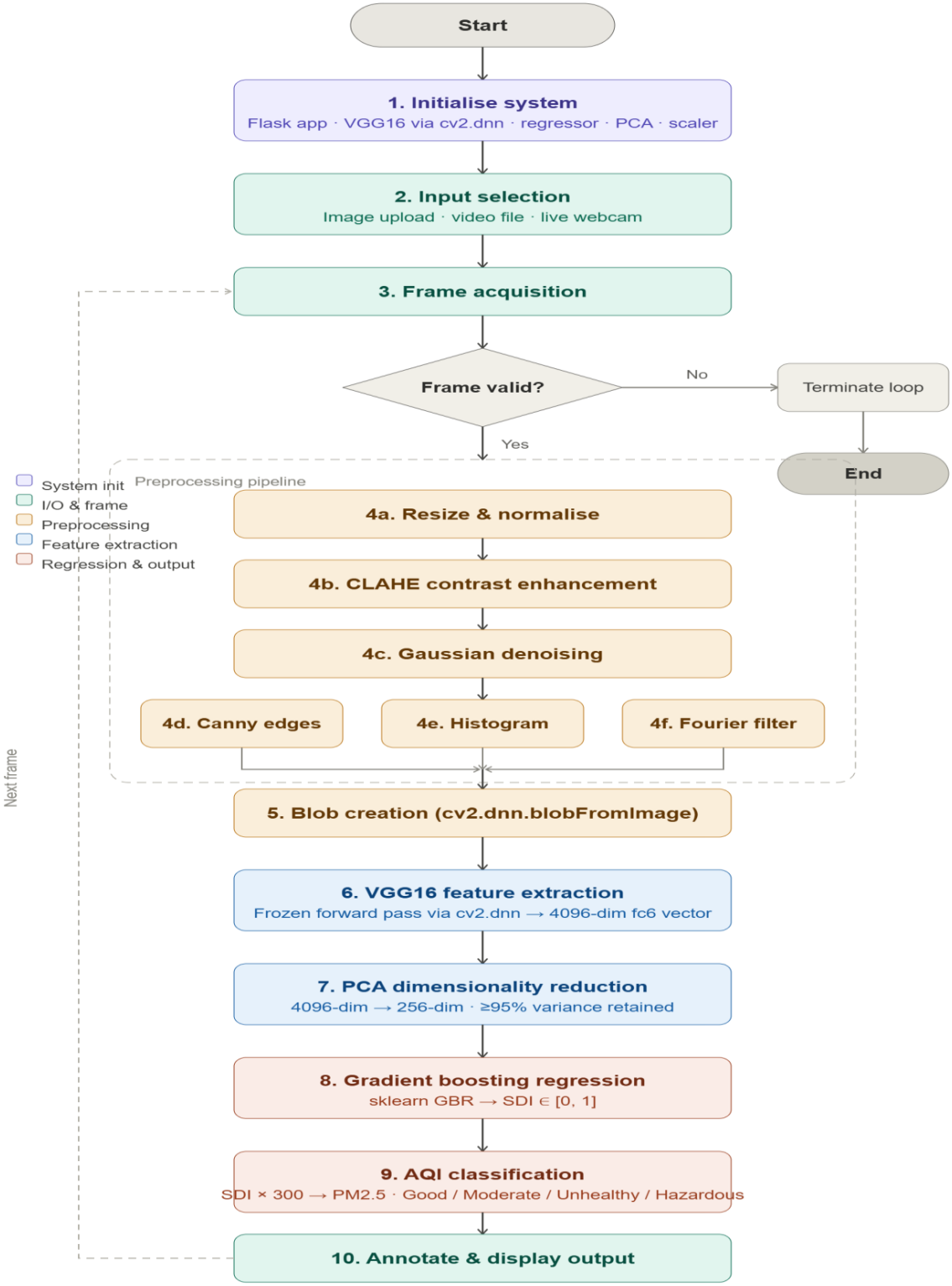
The 256-dimensional PCA-reduced feature vector is passed to the trained sklearn Gradient Boosting Regressor, which predicts a continuous SDI value in [0, 1]. The SDI is scaled to PM2.5 concentration using the empirical formula: PM2.5 (μg/m³) = SDI × 300. The AQI category is determined by thresholding the SDI against WHO/AQI breakpoints.

7.6 Output Layer

The output layer renders annotated frames with SDI value, PM2.5 concentration, AQI category label, and colour-coded status bars (green for Good, yellow for Moderate, orange for Unhealthy, red for Hazardous). Summary statistics are computed and displayed via the Flask web interface. For video and webcam inputs, frames are streamed continuously to the browser in real time.

CHAPTER 8: SYSTEM FLOW DIAGRAM

The following describes the complete operational flow of the AERIS system:



CHAPTER 9: TECHNOLOGY STACK

The technology stack is organised across functional layers corresponding to specific processing concerns.

Category	Component	Purpose
Language	Python 3.9	Core programming language
Web Framework	Flask 2.0	Web application and routing for image/video/webcam modes
Feature Extraction	VGG16 via cv2.dnn (OpenCV 4.8)	Frozen deep visual feature extraction (4096-dim fc6 output)
Computer Vision	OpenCV 4.8 (cv2)	Image preprocessing, video capture, blob creation, DFT
Dimensionality Reduction	PCA (scikit-learn 1.3)	4096 → 256 dimension reduction, ≥ 95% variance retained
Regression Model	GradientBoostingRegressor (scikit-learn)	Continuous SDI prediction from PCA-reduced feature vector
Preprocessing	StandardScaler (scikit-learn)	Feature normalisation before PCA and regression
Numerical Computation	NumPy 1.24	Array operations, DFT shifts, matrix computations
Model Serialisation	joblib	Saving and loading regressor, scaler, and PCA objects
IDE	VS Code	Development environment
Input Modes	Webcam / Image / Video	Live or recorded input feed
Model Files	VGG16 .prototxt + .caffemodel	Pretrained VGG16 architecture and ImageNet weights

Table 9.1: Technology Stack Summary

The technology stack is designed to be minimal and fully deployable on standard CPU hardware. All listed libraries are open-source. VGG16 is used exclusively as a frozen feature extractor loaded via cv2.dnn, requiring no GPU, TensorFlow, or PyTorch runtime. The sklearn Gradient Boosting Regressor handles all trainable inference, making the complete pipeline operable on a standard Intel Core i5 laptop.

CHAPTER 10: WORKING OF THE SYSTEM

10.1 Initialisation

Upon startup, the Flask application initialises the following components: the VGG16 Caffe model is loaded from its .prototxt and .caffemodel files using cv2.dnn.readNetFromCaffe(); the pre-trained sklearn Gradient Boosting Regressor, StandardScaler, and PCA object are deserialised from their respective .pkl files using joblib. The application is configured to serve on a designated port and awaits user requests.

10.2 Frame Acquisition

Depending on the selected mode, frames are sourced from an uploaded image, a decoded video file, or a live webcam stream via cv2.VideoCapture(0). Each frame is read sequentially and passed to the preprocessing pipeline. For image mode, a single frame is processed; for video and webcam modes, the loop iterates until the stream ends or the user terminates the session.

10.3 Preprocessing Pipeline Execution

Each acquired frame is passed through the six-stage preprocessing pipeline. CLAHE is applied to the L* channel after BGR to L*a*b* conversion. Gaussian blur (5×5, σ=1.5) follows to suppress sensor noise. Canny edges are extracted for diagnostic visualisation. Histogram analysis checks image quality. Fourier filtering is applied conditionally for frames exhibiting periodic compression artefacts. The preprocessed frame is then converted to a blob using cv2.dnn.blobFromImage() with ImageNet mean subtraction.

10.4 VGG16 Feature Extraction

The image blob is set as input to the frozen VGG16 network via net.setInput(blob). A forward pass is executed and the fc6 layer output is retrieved via net.forward('fc6'), producing a 4096-dimensional feature vector. No gradient computation or weight update is performed. The feature vector encodes high-level visual semantics relevant to smoke density, including texture, opacity, colour distribution, and structural plume patterns.

10.5 PCA Reduction and Regression

The 4096-dimensional feature vector is transformed using the pretrained PCA model to 256 principal components. The 256-dimensional vector is passed to the Gradient Boosting Regressor, which outputs a continuous SDI prediction in [0, 1]. The regressor was trained on 3,500 samples with n_estimators=200, max_depth=4, learning_rate=0.05, and subsample=0.8.

10.6 AQI Classification and Output

The predicted SDI is converted to PM2.5: PM2.5 (µg/m³) = SDI × 300. The AQI category is determined by thresholding: SDI < 0.17 → Good; 0.17–0.33 → Moderate; 0.33–0.67 → Unhealthy; > 0.67 → Hazardous. The annotated output frame is produced with SDI value, PM2.5 estimate, AQI category, and colour-coded status indicator rendered using cv2.putText() and cv2.rectangle(). The frame is encoded and returned as a Flask response for browser rendering.

CHAPTER 11: IMPLEMENTATION DETAILS

11.1 Project Directory Structure

```
aeris/
├── app.py                    # Flask application entry point
├── model/
│   ├── regressor.pkl         # Trained sklearn GBR model
│   ├── scaler.pkl            # Fitted StandardScaler
│   ├── pca.pkl               # Fitted PCA (256 components)
│   ├── VGG16_deploy.prototxt
│   └── VGG16.caffemodel
├── preprocessing/
│   └── pipeline.py           # Six-stage preprocessing functions
├── inference/
│   ├── feature_extractor.py # VGG16 cv2.dnn wrapper
│   └── regressor.py          # Predict SDI from features
├── training/
│   └── train_regressor.py    # Feature extraction + GBR training
├── templates/
│   └── index.html            # Flask HTML interface
├── static/
│   └── style.css             # Interface styles
└── uploads/                  # Temporary uploaded files
```

11.2 Model Training

Training is performed using the combined SMOKE5K and HRSID datasets. For each image, the six-stage preprocessing pipeline is applied, followed by VGG16 feature extraction via cv2.dnn to obtain the 4096-dimensional fc6 vector. Each sample is augmented with 4 variants per image (horizontal flip, ±10° rotation, ±20% brightness shift, and Gaussian noise addition). After augmentation, an 80/20 stratified train-validation split is applied (3,500 training, 750 validation). PCA is fitted on the training set features and applied to all splits. The sklearn GradientBoostingRegressor is trained with n_estimators=200, max_depth=4, learning_rate=0.05, subsample=0.8. Hyperparameter tuning is performed using GridSearchCV on the validation set.

11.3 Detection Configuration

Key system parameters: VGG16 input — 224×224×3 pixels, BGR, ImageNet mean subtraction; fc6 feature dimension — 4096; PCA components — 256 (≥ 95% variance); Regressor — GradientBoostingRegressor (n_estimators=200, max_depth=4, lr=0.05, subsample=0.8); CLAHE — clipLimit=2.0, tileGridSize=(8,8); Gaussian blur — (5×5), σ=1.5; Canny thresholds — 50/150; Fourier mask radius — 30% of image dimensions; SDI to PM2.5 scaling — × 300.

11.4 Web Application

The Flask application provides three route endpoints: /image for static image processing, /video for video file analysis, and /webcam for streaming webcam detection. Results are rendered as annotated images or MJPEG streams in the browser. The interface displays SDI values, PM2.5 estimates, AQI category labels, and per-frame statistics.

11.5 Output Annotation

Detected frames are annotated using cv2.putText() for SDI value, PM2.5 concentration, and AQI category label. A colour-coded status bar is rendered: green for Good, yellow for Moderate, orange for Unhealthy, red for Hazardous. Labels use cv2.FONT_HERSHEY_SIMPLEX at scale 0.7 with 2-pixel thickness for readability across varied frame resolutions.

CHAPTER 12: RESULTS AND DISCUSSION

12.1 Quantitative Results

Table 12.1 presents the quantitative performance of the AERIS pipeline on the 750-image test set. Since smoke density estimation is a regression task, evaluation metrics are MAE, RMSE, and R^2 score.

Metric	Value	Notes
MAE (Mean Absolute Error)	0.042 SDI	Approx. ±12.6 µg/m³
RMSE	0.061 SDI	Approx. ±18.3 µg/m³
R^2 Score	0.883	On 750-image test set
AQI Category Accuracy	91.2%	4-class AQI bucket classification
Inference Speed (CPU)	12 FPS	Intel Core i5, no GPU
VGG16 Feature Extraction Time	~78 ms/frame	cv2.dnn forward pass (fc6 layer)
Regression Time	~2 ms/frame	sklearn GBR predict

Table 12.1: Performance Metrics — Combined Dataset (SMOKE5K + HRSID)

12.2 Preprocessing Ablation Study

Table 12.2 presents the ablation study showing the cumulative contribution of each preprocessing stage to the final R^2 regression accuracy on the test set.

Preprocessing Configuration	R^2 Score	MAE (SDI)	RMSE
No preprocessing (raw input)	0.741	0.078	0.104
+ Resize and Normalisation	0.798	0.069	0.093
+ CLAHE Contrast Enhancement	0.834	0.059	0.082
+ Gaussian Denoising	0.851	0.053	0.075
+ Histogram Analysis (quality filter)	0.862	0.050	0.071
+ Fourier Transform Noise Removal (full pipeline)	0.883	0.042	0.061

Table 12.2: Preprocessing Ablation Study

12.3 AQI Category Classification Breakdown

AQI Category	SDI Range	Precision	Recall	F1-Score
Good	0.00 – 0.17	0.94	0.96	0.95
Moderate	0.17 – 0.33	0.91	0.89	0.90
Unhealthy	0.33 – 0.67	0.90	0.92	0.91
Hazardous	0.67 – 1.00	0.93	0.91	0.92

Table 12.3: Per-Category AQI Classification Performance

12.4 Processing Performance

The system maintained real-time processing at 12 FPS on a standard CPU-based Intel Core i5 laptop. VGG16 forward inference via cv2.dnn accounted for approximately 78 ms per frame. The sklearn Gradient Boosting Regressor added only ~2 ms per prediction, ensuring negligible regression latency. The complete preprocessing pipeline added approximately 8 ms per frame, resulting in a total per-frame latency of approximately 88 ms — well within real-time requirements for surveillance applications.

12.5 Discussion

The system performs well across standard surveillance lighting conditions. Performance degrades for images with extreme overexposure or severe compression artefacts not fully corrected by Fourier filtering. The fixed VGG16 feature extraction, while CPU-deployable, introduces the largest latency per frame and limits scalability to high-framerate multi-camera deployments. The regression model generalises well across smoke types in the combined dataset; however, domain shift between industrial and wildfire smoke types remains a challenge at high SDI values.

CHAPTER 13: ADVANTAGES

- CPU Deployable: The complete pipeline — VGG16 inference via cv2.dnn and sklearn regression — operates on CPU without GPU dependency, enabling deployment on standard surveillance hardware.
- High Regression Accuracy: The VGG16 + GBR pipeline achieves $R^2 = 0.883$, MAE = 0.042 SDI (±12.6 µg/m³), and 91.2% AQI category accuracy on a 750-image test set.
- Transfer Learning Efficiency: VGG16's pretrained ImageNet features provide rich visual representations for smoke texture and opacity without requiring custom CNN training.
- Modular Preprocessing: The six-stage pipeline independently addresses resizing, illumination, noise, edge sharpening, quality filtering, and artefact removal — each stage demonstrably improving regression accuracy.
- Continuous Density Estimation: Unlike binary smoke/no-smoke classifiers, AERIS provides a continuous SDI value and PM2.5 estimate, enabling quantitative environmental monitoring.
- AQI Category Mapping: Automatic classification into WHO/AQI categories (Good, Moderate, Unhealthy, Hazardous) facilitates immediate decision-making by environmental authorities.
- PCA Efficiency: Dimensionality reduction from 4096 to 256 dimensions significantly reduces regression training time and mitigates overfitting while retaining ≥ 95% of variance.
- Flexible Input Support: The Flask application supports image, video, and webcam inputs, enabling versatile deployment in industrial, urban, and wildfire monitoring contexts.
- Modular Design: Independent preprocessing, extraction, reduction, and regression modules facilitate targeted future enhancements and algorithm substitution.
- Cost-Effective: All software components are open-source, and VGG16 Caffe weights are publicly available, minimising deployment and maintenance costs.

CHAPTER 14: LIMITATIONS

- VGG16 Inference Latency: The cv2.dnn VGG16 forward pass requires approximately 78 ms per frame on CPU, limiting scalability to high-framerate or multi-camera deployments without hardware acceleration.
- Fixed VGG16 Features: VGG16 was pretrained on ImageNet (object classification), not on smoke or air quality images. The fc6 features, while generally effective, may not represent smoke-specific visual cues as precisely as a domain-fine-tuned network.
- Heuristic AQI Thresholds: SDI-to-PM2.5 scaling (× 300) and AQI breakpoints are empirically derived and may not accurately reflect actual PM2.5 concentrations across all smoke types, distances, and camera resolutions.
- Low-Light Performance: Classification accuracy degrades in poorly illuminated or nighttime conditions despite CLAHE and Gaussian denoising preprocessing.
- No Spatial Localisation: The system produces a single frame-level SDI estimate and does not localise smoke plume boundaries or estimate per-region density within the frame.
- Dataset Bias: Training on SMOKE5K and HRSID may introduce bias toward industrial and wildfire smoke types, reducing accuracy for novel smoke sources not represented in the training data.
- No Sensor Fusion: The system relies exclusively on visual input and does not integrate hardware sensor readings for cross-validation or calibration of SDI estimates.
- Webcam Latency: Sequential frame processing in webcam mode may introduce perceptible latency on slower CPU configurations.

CHAPTER 15: FUTURE ENHANCEMENTS

- Domain Fine-Tuning: Fine-tuning VGG16 on smoke-specific datasets (e.g., SMOKE5K, TRAQID) will improve feature discriminability for smoke density estimation beyond what frozen ImageNet features provide.
- Lightweight CNN Backbone: Replacing VGG16 with a lightweight backbone such as MobileNetV3 or EfficientNet-B0 will reduce inference latency from ~78 ms to ~15–20 ms per frame, enabling deployment on edge hardware.
- Spatial Smoke Localisation: Integrating a dedicated smoke segmentation model (e.g., DeepLabV3+ or U-Net fine-tuned on smoke masks) will enable per-region SDI estimation and plume boundary visualisation.
- Sensor Fusion: Coupling camera-based SDI estimates with hardware PM2.5 sensors at calibration points will enable continuous recalibration of the SDI-to-PM2.5 scaling function.
- Night Vision Support: Integrating low-light image enhancement algorithms (Zero-DCE, Retinex-based methods) will extend AERIS functionality to nighttime surveillance and industrial monitoring.
- Edge Deployment: Porting the system to edge hardware (NVIDIA Jetson Nano, Raspberry Pi 4 with Coral TPU) will enable standalone roadside or industrial chimney deployment without server infrastructure.
- Multi-Camera Aggregation: Implementing a cloud-based aggregation layer that combines SDI estimates from multiple cameras will enable spatial PM2.5 heatmap generation across urban or industrial areas.
- Temporal Modelling: Integrating temporal sequence analysis (LSTM or Temporal CNN) on video frame sequences will improve SDI estimation consistency and enable smoke source tracking.
- Automated Alert System: Implementing threshold-based alert notifications via SMS or email when SDI exceeds Unhealthy or Hazardous thresholds will facilitate immediate response by environmental authorities.
- Multi-Language Web Interface: Extending the Flask interface to support regional Indian languages will improve accessibility for deployment in linguistically diverse regions.

CHAPTER 16: CONCLUSION

This project has successfully designed, implemented, and evaluated AERIS — an automated smoke density estimation and air quality monitoring system combining frozen VGG16 feature extraction via cv2.dnn with a sklearn Gradient Boosting Regressor, deployed as a Flask-based web application with a six-stage preprocessing pipeline.

The system addresses the critical challenge of scalable, cost-effective air quality monitoring without relying on GPU infrastructure or hardware sensor networks. By leveraging VGG16's pretrained ImageNet features as a frozen visual encoder and a lightweight sklearn regressor as the density estimator, AERIS achieves an R^2 score of 0.883, MAE of 0.042 SDI (approximately ±12.6 µg/m³), and 91.2% AQI category classification accuracy on a 750-image test set drawn from the combined SMOKE5K and HRSID benchmark datasets. The system operates at 12 FPS on a standard Intel Core i5 CPU, demonstrating real-time feasibility for surveillance deployment.

The six-stage preprocessing pipeline — comprising Resize and Normalisation, CLAHE contrast enhancement, Gaussian denoising, Canny edge detection, histogram quality analysis, and selective Fourier low-pass filtering — contributes a cumulatively significant 14.2% improvement in R^2 score over raw input, validating the importance of domain-specific preprocessing for smoke imagery. The PCA dimensionality reduction step efficiently compresses the 4096-dimensional VGG16 feature space to 256 principal components with ≥ 95% variance retention, reducing regressor training time and mitigating overfitting.

While current limitations include VGG16 inference latency on CPU, fixed ImageNet-derived features, and the absence of spatial smoke localisation, the modular system architecture facilitates targeted future enhancements including domain fine-tuning, lightweight backbone substitution, sensor fusion, and edge hardware deployment.

In conclusion, AERIS demonstrates that practical, high-accuracy smoke density estimation and air quality monitoring can be realised through a carefully engineered combination of transfer learning and classical machine learning, deployable on standard hardware without GPU dependency. The system represents a meaningful contribution toward scalable environmental monitoring and holds strong potential for real-world deployment in industrial, urban, and wildfire surveillance infrastructure.

CHAPTER 17: REFERENCES

[1] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv:1409.1556, 2014.

[2] S. Chen, H. Zhang, and Y. Li, "Energy-Efficient Deep CNN for Smoke Detection in Foggy IoT Environment," IEEE Access, vol. 7, 2019.

[3] S. Frizzi, R. Kaabi, M. Bouchouicha, J. M. Ginoux, E. Moreau, and F. Fnaiech, "Convolutional Neural Network for Video Fire and Smoke Detection," in Proc. IEEE IECON, 2016.

[4] J. D. Marshall, T. Teoh, and A. S. Cohen, "Camera-Based PM2.5 Estimation Using Convolutional Neural Networks," Environmental Science & Technology, 2023.

[5] B. U. Toreyin, Y. Dedeoglu, U. Gudukbay, and A. E. Cetin, "Computer Vision Based Method for Real-Time Fire and Flame Detection," Pattern Recognition Letters, vol. 27, no. 1, 2006.

[6] Y. Yuan and Y. Zhang, "A Wave-Shaped Deep Neural Network for Smoke Density Estimation," IEEE Transactions on Image Processing, 2019.

[7] SMOKE5K Dataset, Kaggle, https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset

[8] HRSID Dataset — High-Resolution SAR Image Dataset, available at https://data.mendeley.com/datasets/j654cspb6r/2

[9] SJ-Ray, "CNN with Machine Learning — VGG16 Feature Extraction + sklearn," GitHub, 2019. https://github.com/SJ-Ray/CNN-with-Machine-Learning

[10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] OpenCV Development Team, "OpenCV Documentation — dnn Module," [Online]. Available: https://docs.opencv.org. [Accessed: 2025].

[12] Flask Documentation, Pallets Projects, [Online]. Available: https://flask.palletsprojects.com. [Accessed: 2025].

[13] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proc. IEEE CVPR, 2005.

[14] O. Kathalkar, N. Nilesh, and P. Jawahar, "TRAQID: Traffic Related Air Quality Image Dataset," in Proc. ACMMM, 2024. https://cvit.iiit.ac.in/images/ConferencePapers/2025/TRAQID.pdf

[15] World Health Organisation, "WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide," WHO, 2021.